

Fast National University of computing and Emerging Sciences Peshawar Campus



Data Structures

Practice Problem(Stack using Linked list)

1. Implement a Stack using Linked List:

Write a C++ program to implement a basic stack using a singly linked list.

2. Push and Pop Operations:

Implement the push and pop operations for a stack using a linked list.

3. Stack Overflow and Underflow:

Write a C++ program that detects stack overflow and underflow conditions for a linked list implementation.

4. Peek/Top Operation:

Implement a function to return the top element of the stack without removing it from the stack.

5. Reverse a String using Stack:

Write a program to reverse a string using a stack implemented with a linked list.

6. Check for Balanced Parentheses:

Using a stack, write a program to check if an expression has balanced parentheses.

7. Convert Infix to Postfix using Stack:

Implement a program to convert an infix expression to postfix notation using a stack.

8. Evaluate Postfix Expression using Stack:

Implement a C++ program to evaluate a postfix expression using a stack.

9. Find Minimum Element in a Stack:

Modify the stack implementation to find the minimum element in constant time during push and pop operations.

10. Reverse a Linked List using Stack:

Write a program to reverse a singly linked list using a stack data structure.

11. Sort a Stack:

Implement a program to sort a stack using recursion or another stack.

12. Sort a Stack with Linked List:

Modify the stack implemented using a linked list to support sorting while maintaining the stack order.

13. Check Palindrome using Stack:

Write a C++ program to check whether a given string is a palindrome using a stack implemented with a linked list.

14. Find Middle Element in a Stack:

Implement a function to find the middle element of a stack in constant time.

15. Implement a Two-Stack Queue:

Write a program that uses two stacks (implemented with linked lists) to create a queue with enqueue and dequeue operations.

16. Clone a Stack:

Write a C++ program to clone an existing stack implemented with a linked list.

17. Sort Multiple Stacks:

Given multiple stacks implemented with linked lists, write a program to sort all stacks individually.

18. Reverse Stack using Recursion:

Implement a function to reverse a stack using recursion, but without using any extra stack or data structure.

19. Detect Stack Cycles:

Modify your stack implementation to detect cycles or loops within the linked list representing the stack.

20. Stack with Duplicate Entries:

Implement a stack using a linked list where duplicate elements are allowed, and create a function that removes duplicates from the stack.